W pełni lokalny, offline pipeline mowy dla Unitree G1: ASR → LLM → TTS na Jetson Orin NX

Streszczenie kierownicze

Unitree G1 ma rozdzieloną architekturę sterowania: komputer do lokomocji jest przeznaczony do programu kontroli ruchu Unitree i nie jest publicznie otwarty, a wszelka „wtórna" logika aplikacyjna (w tym przetwarzanie mowy) powinna działać na komputerze developerskim. To naturalny i bezpieczny wzorzec: **ASR/LLM/TTS jako warstwa intencji i dialogu**, nigdy jako element twardej pętli sterowania.

Z dokumentacji Unitree wynika, że komputer developerski (PC2) w G1 jest oparty o **NVIDIA Jetson Orin NX** (8 rdzeni CPU Arm Cortex-A78AE, 16 GB RAM, GPU Ampere 1024 CUDA/32 Tensor) oraz ma domyślny adres IP 192.168.123.164. To czyni w pełni lokalny pipeline mowy wykonalnym, ale wymusza rygor w zarządzaniu zasobami (RAM/GPU/termika) i w projektowaniu bezpieczeństwa.

Rekomendowany stos (praktyczny „offline-first", ARM64/Jetson, szybkie wdrożenie) to:

- **VAD**: Silero VAD w ONNX (MIT) jako główny detektor mowy (lepsza jakość niż proste VAD energetyczne, szeroka przenośność ONNX).
- **ASR**: 2-torowo zależnie od priorytetu:
    - **Faster-Whisper (CTranslate2)** (MIT) + modele Whisper (int8/float16) dla najlepszej relacji jakość↔szybkość na GPU/CPU; projekt deklaruje do ~4× względem referencyjnego openai/whisper i wsparcie 8-bit quant.
    - **Vosk** (Apache-2.0) jako bardzo lekka, strumieniowa alternatywa o małych modelach (~50 MB) i niskiej latencji na słabszym CPU, z modelem PL; dobry „tryb awaryjny", gdy GPU jest zajęte wizją/LLM.
    - (Opcjonalnie, gdy chcesz minimalizmu C/C++): **whisper.cpp** z kwantyzacją ggml (int-quant) i znanymi rozmiarami pamięci; koszt to częściej większa praca integracyjna i ryzyko długiego cold-start na niektórych Jetsonach.
- **LLM**: mały model lokalny 2–4B w int4 (np. Qwen3.5-2B/4B), z krótkim kontekstem i wyłączonym „thinking mode" dla minimalizacji latencji; Qwen3.5 domyślnie generuje „thinking content", ale dokumentuje tryb non-thinking i serwowanie przez popularne runtime'y.
- **TTS**:
    - **Piper (MIT)** jako domyślny neuralny TTS offline, z gotowymi binarkami arm64 oraz z szeroką bazą głosów (w tym pl_PL); typowe modele „medium" mają rozmiar rzędu dziesiątek MB (np. ~63 MB dla pl_PL-gosia-medium), a jakość jest stopniowana.
    - **eSpeak NG (GPL-3.0+)** jako ultralekki fallback „zawsze działa" (niski koszt CPU, duża liczba języków), ale z istotnym wpływem licencji GPL na dystrybucję.

- (opcjonalnie enterprise): **NVIDIA Riva** – mikroserwisy ASR/TTS przyspieszane na GPU, z trybem offline i streaming, oraz jawnie podanymi wymaganiami pamięci i wsparciem wybranych modeli na urządzeniach embedded.

W praktyce, na Orin NX 16GB w robocie najczęściej opłaca się celować w budżet: **„czas do pierwszego dźwięku odpowiedzi" 0.7–2.5 s** dla typowych komend (1–4 s mowy użytkownika), przy jednoczesnym utrzymaniu percepcji/wizji i stabilnych limitów termicznych. Ten budżet jest osiągalny przy strumieniowym ASR + non-thinking LLM + strumieniowym TTS i agresywnych limitach kontekstu oraz time-outach.

Założenia sprzętowe Unitree G1 i implikacje dla audio

Komputer developerski i granice integracji

Unitree opisuje, że jednostka obliczeniowa do transportu/kontroli (lokomocja) jest przeznaczona do programu kontroli Unitree i nie jest otwarta dla deweloperów; deweloperzy mogą używać tylko jednostki developerskiej do „secondary development". To praktycznie narzuca architekturę, w której **ASR/LLM/TTS uruchamiasz wyłącznie na komputerze developerskim** i komunikujesz się z kontrolą ruchu przez dozwolone API (DDS/ROS2-bridge, zależnie od SDK).

Parametry referencyjne sprzętu

Na stronie Unitree (developer) podano parametry „development computing unit (PC2)" dla G1: Jetson Orin NX, CPU Arm Cortex-A78AE (8 rdzeni), RAM 16G, GPU Ampere 1024 CUDA/32 Tensor, oraz wskazano IP 192.168.123.164; widnieje też „Storage 2T" (interpretuj jako pojemność magazynu danych w tej konfiguracji).

Po stronie platformy Jetson Orin NX 16GB datasheet wskazuje m.in. tryby mocy 10W/15W/25W, wsparcie zasilania 5–20V oraz możliwość użycia zewnętrznego NVMe przez PCIe (x2/x4). W kontekście audio to ważne, bo:

- ASR i TTS w praktyce konkurują o RAM z LLM i ewentualnie z pipeline'em wizyjnym.
- Stabilna latencja jest silnie zależna od termiki i profilu mocy (unikanie throttlingu).
- Trzymanie wielu modeli (ASR/TTS/LLM) jest realne przy dużym magazynie, ale **cold-start** i I/O muszą być kontrolowane.

Sieć i tryb offline

Przewodniki integracyjne G1 opisują statyczną sieć wewnętrzną i sugerują zmianę domyślnych haseł, gdy robot ma mieć kontakt z Internetem. Dla trybu **w pełni offline** jest to korzystne: utrzymujesz szczelną segmentację i minimalizujesz powierzchnię ataku, a jednocześnie musisz zaplanować **offline-updates** (nośnik, repozytorium artefaktów, podpisy).

Offline ASR na Jetson Orin: rekomendacje i tabela porównawcza

Jak dobierać ASR dla robota humanoidalnego

W robotyce użytkowej (komendy, rozmowa w ruchu) ASR musi łączyć cztery cechy: niska latencja (partial results), odporność na hałas i pogłos, stabilność strumieniowa oraz przewidywalny koszt zasobów. Model Whisper (OpenAI) jest silny jakościowo i trenowany w skali 680k godzin danych wielojęzycznych i multitask, co poprawia odporność m.in. na szum i akcenty. Jednocześnie Whisper bywa cięższy, a modele większe zwiększają latencję i utrudniają wdrożenia na urządzeniach o ograniczonych zasobach (to jest też obiekt badań pod kątem kwantyzacji i wdrażania).

W praktyce polecam **dwa tryby ASR**:

- **Tryb jakości**: faster-whisper (CTranslate2) z modelem Whisper w int8/FP16, z streaming policy (partial), gdy GPU ma headroom.
- **Tryb deterministycznie lekki**: Vosk (małe modele, streaming), gdy GPU jest obciążone (wizja/LLM) albo gdy chcesz maksymalnie niskiej mocy/temperatury.

Tabela porównawcza ASR

W tabeli rozdzielam „silnik" (engine) od „wag" (modele). W przypadku Whisper rozmiary modeli podaję na podstawie danych z whisper.cpp (dysk/pamięć) jako praktycznej referencji wdrożeniowej.

Kandydat ASR	Licencja silnika	Modele i rozmiary	Kwantyzacja / formaty	Typowe wymagania (Jetson Orin NX 16GB – rząd wielkości)	Latencja i tryb pracy	Frameworki / integracja	Dojrzałość i społeczność
Faster-Whisper (CTranslate2)	MIT (pakiet i repo)	Wagi Whisper (tiny…large). W ekosystemie typowe konwersje do CTranslate2 z int8.	CTranslate2 wspiera quant (m.in. int8_*). Dokumentacja opisuje mechanikę kwantyzacji.	RAM/VRAM zależne od wariantu i batch; int8 znacząco poprawia footprint. Faster-whisper deklaruje mniejsze zużycie pamięci i do ~4× szybkości vs openai/whisper, oraz poprawę dzięki 8-bit quant na CPU i GPU.	Dobre do streaming (przez otoczkę/politykę segmentacji) i offline. Repo wskazuje też ekosystem narzędzi streamingowych używających faster-whisper jako backend.	Python + CTranslate2 (C++), CUDA/CPU, łatwy Docker.	Bardzo wysoka adopcja (Whisper ecosystem), stabilny runtime produkcyjny (CT2).
whisper.cpp	MIT	Modele Whisper w formacie ggml/gguf. Repo publikuje orientacyjne rozmiary: np. tiny ~75 MiB dysk/~273 MB RAM; base ~142 MiB/~388 MB; small ~466 MiB/~852 MB; medium ~1.5 GiB/~2.1 GB; large ~2.9 GiB/~3.9 GB.	Wspiera integer quantization modeli ggml (np. Q5_0 itd.) i dostarcza narzędzia do kwantyzacji.	CPU-friendly, opcjonalnie CUDA. Dla małych modeli realne uruchomienie nawet bez GPU, ale przy jakości/hałasie lepiej mieć headroom. Ryzyko: raportowane przypadki bardzo długiego czasu ładowania na niektórych Jetsonach (cold-start bottleneck).	Ma tryb „stream" – przykład narzędzia próbkowania audio co 0.5 s i ciągłej transkrypcji (to daje naturalny rytm partial results).	C/C++ (ggml), proste wiązania; bardzo dobre do integracji w systemy wbudowane.	Bardzo wysoka dojrzałość (częste releasy), ogromna społeczność.
Vosk	Apache-2.0 (repo i strona)	Modele „small" rzędu ~50 MB (deklaracja w PyPI) i większe dla jakości; dostępne modele PL.	Brak „typowego" int4/int8 jak w transformerach; optymalizacja przez dobór	Bardzo niski koszt pamięci (dziesiątki MB + narzut), dobra praca na CPU; nadaje się, gdy GPU jest zajęte.	Streaming API; PyPI opisuje „zero-latency response with streaming API" (w praktyce: szybkie partial results).	Wiele bindingów: Python/Java/C#/C++/Node i inne.	Wysoka dojrzałość wdrożeniowa (IoT/RPi), szeroka społeczność.

Kandydat ASR	Licencja silnika	Modele i rozmiary	Kwantyzacja / formaty	Typowe wymagania (Jetson Orin NX 16GB – rząd wielkości)	Latencja i tryb pracy	Frameworki / integracja	Dojrzałość i społeczność
			małych modeli i dekodera.				
sherpa-onnx (STT)	Apache-2.0	Zależne od wybranego modelu ONNX (CTC/RNNT itd.); projekt jest frameworkiem deploymentowym.	ONNX Runtime (int8 możliwy zależnie od modelu i pipeline'u); często łatwiej o akcelerację niż w „czystym" PyTorch na edge.	Footprint zależny od modelu; duża wartość: jedna biblioteka obejmuje STT+TTS+VAD+diaryzację offline.	Wprost promuje działanie „without Internet connection" i offline aplikacje.	C++/Python i wiele platform embedded (wg opisu projektu).	Średnio-wysoka dojrzałość; mniejsza ekosystemowo niż Whisper, ale mocna jako „szwajcarski scyzoryk" ONNX.
NVIDIA Riva (opcjonalnie)	Komercyjny SDK/mikroserwisy NVIDIA	Modele zależne od pakietu Riva; support matrix podaje też, które modele są wspierane na embedded.	GPU-accelerated; liczne tryby offline/streaming (ASR, TTS).	Support matrix podaje liczby: np. ASR conformer-ctc ma GPU mem ~3.6 GB (streaming) do ~5.8 GB (offline) i jest oznaczony jako wspierany na embedded; z kolei whisper-large-v3 w Riva nie jest wspierany na embedded (wysokie VRAM).	Mikroserwisy pod real-time; dobre gdy akceptujesz cięższy stack i licencję NVIDIA.	gRPC, kontenery (typowo), GPU.	Wysoka dojrzałość enterprise; mniejsza elastyczność licencyjna/offline-update w porównaniu do MIT/Apache.

Offline TTS na Jetson Orin: rekomendacje i tabela porównawcza

Jak dobierać TTS dla robota

W humanoidzie TTS ma inny profil niż ASR: ważniejsze są (a) „czas do pierwszego dźwięku" (TTST – time-to-start-talking), (b) stabilność streamingu i brak glitchy audio, (c) kontrola prozodii/intonacji w ograniczonych zasobach oraz (d) licencja głosu i zależności.

W praktyce opłaca się zastosować **dwie warstwy TTS**:

- Neuralny TTS główny (naturalność): Piper (VITS-like) w ONNX, z polskim głosem (np. gosia/justyna/jarvis_wg_glos itd.).
- Fallback ultralekki: eSpeak NG (formant) – działa niemal zawsze, ale jest GPL-3.0+ (konsekwencje licencyjne dystrybucji).

Jeśli potrzebujesz ścieżki „enterprise" (np. twarde SLA, gotowe mikroserwisy, integracja GPU): Riva TTS oferuje streaming i offline inference, a support matrix wskazuje np. TTS fastpitch-hifigan jako wspierany na embedded z GPU mem ~2 GB.

Tabela porównawcza TTS

Kandydat TTS	Licencja silnika	Głosy / modele PL (przykłady) i rozmiary	Kwantyzacja / formaty	Wymagania (Orin NX – rząd wielkości)	Latencja i tryb	Frameworki / integracja	Dojrzałość i ryzyka
Piper	MIT	Oficjalne piper-voices (MIT) zawiera pl_PL głosy; np. pl_PL-gosia-medium ma ~63.2 MB (ONNX).	ONNX (runtime). Jakość ma poziomy: x_low (5–7M parametrów), low (15–20M), medium (15–20M), high (28–32M).	Zwykle CPU-friendly; RAM od setek MB (zależnie od modelu i buforów). Na Jetson często opłaca się trzymać TTS na CPU, zostawiając GPU dla LLM/wizji. (Estymacja inżynierska, do potwierdzenia benchmarkiem).	Bardzo dobry do streamingu „prawie natychmiast": TTST typowo dziesiątki–setki ms przy małych buforach; prędkość generacji zwykle >1× real-time na współczesnych CPU dla małych modeli (zależne od głosu).	Binarki arm64 dostępne; można używać jako proces CLI lub biblioteka.	Bardzo wysoka popularność. Ryzyko licencyjne w praktyce dotyczy zależności fonemizera: w dyskusjach wskazuje się użycie espeak-ng w piper-phonemizer (GPL), co trzeba przeanalizować pod kątem dystrybucji.
sherpa-onnx (TTS)	Apache-2.0 (silnik)	Strona projektu publikuje listy polskich modeli TTS (np. vits-piper-pl_PL-gosia-medium itd.).	ONNX Runtime; format zależny od modelu (często VITS-style).	Podobne do Piper (bo często te same rodziny głosów); plus wartość: spójny stack offline (STT+TTS+VAD+diaryzacja).	W zależności od implementacji potrafi robić streaming i offline; dobry gdy chcesz jeden SDK zamiast wielu komponentów.	C++/Python + ONNX Runtime.	Średnio-wysoka dojrzałość, mniejsza „mainstreamowość" niż Piper, ale mocne zalety integracyjne.
Coqui TTS (toolkit)	Kod: MPL-2.0 (komercyjnie dozwolone), modele mogą mieć inne licencje	Możliwość użycia VITS/HiFi-GAN itd. (toolkit), ale polskie, gotowe, „bezpiecznie komercyjne" głosy trzeba zweryfikować per-model.	W praktyce: PyTorch; kwantyzacja możliwa przez ONNX/TensorRT, ale nie jest „no-brainer" jak w Piper.	Zależne od modelu; XTTS/klonowanie jest zwykle cięższe niż Piper i bywa słabe licencyjnie do produktu.	Może mieć świetną jakość, ale TTST i zasoby zależą od architektury (VITS vs Tacotron2+vocoder).	PyTorch, Docker, pipeline'y treningowe.	Ryzyko: część popularnych modeli (np. XTTS-v2) jest pod CPML, który wprost ogranicza do użytku niekomercyjnego; w dyskusjach społeczność stwierdza brak ścieżki pozyskania licencji komercyjnej po zamknięciu firmy.
NVIDIA Riva TTS (opcjonalnie)	Komercyjny SDK NVIDIA	Modele TTS w Riva; support matrix wskazuje TTS fastpitch-hifigan jako wspierany na embedded (GPU mem ok. 2 GB).	GPU-accelerated; tryb streaming i offline.	Wymagania pamięci jawne w support matrix.	Streaming: audio zwracane w trakcie syntezy; offline: zwraca po wygenerowaniu całości (większa przepustowość).	Mikroserwisy (typowo kontenery), gRPC.	Bardzo dobra jakość i wydajność, ale koszty operacyjne (kontenery, wersjonowanie) i licencja.
eSpeak NG (fallback)	GPL-3.0+	Języki liczne, w tym europejskie; jakość „robotyczna", ale stabilność i niski koszt zasobów.	Nie dotyczy (synteza formantowa).	Bardzo niski koszt CPU/RAM, praktycznie zawsze działa.	Natychmiastowy start mówienia (TTST minimalny).	Biblioteki/CLI, prosta integracja.	Kluczowe ryzyko: GPL w produkcie wbudowanym (dystrybucja).

Projekt pipeline'u offline ASR → LLM → TTS

Przepływ danych, buforowanie i współbieżność

Zalecany projekt to **pipeline strumieniowy**, gdzie audio jest przetwarzane w stałych ramkach, a system wytwarza *partial transcripts* i *partial audio* w sposób ciągły. To minimalizuje „ciszę po wypowiedzi użytkownika" i poprawia UX.

Kluczowe etapy:

- **Audio capture**: 16 kHz mono PCM (spójnie dla VAD/ASR; whisper.cpp i Silero VAD są często używane w 16 kHz).
- **VAD**: detekcja start/stop mowy, „hangover" (np. 200–400 ms ciszy zanim uznasz koniec wypowiedzi) oraz filtr krótkich szumów.
- **ASR streaming**:
    - tryb „partial" co N ms (np. 200–500 ms),
    - tryb „final" po końcu mowy (EoU – end-of-utterance).
    - W whisper.cpp istnieje podejście demonstracyjne, gdzie narzędzie strumieniowe próbuje audio co 0.5 s i robi transkrypcję ciągłą – to jest dobry punkt odniesienia dla interwału partial.
    - Vosk wprost promuje streaming API o niskiej latencji i małych modelach.
- **Normalizacja tekstu**: usuwanie wtrąceń, detekcja języka (jeśli potrzebujesz), opcjonalna interpunkcja. Whisper ma zwykle lepszą interpunkcję „z pudełka", bo jest multitask; Vosk często wymaga dodatkowej obróbki (np. model interpunkcji albo heurystyki).
- **LLM**: przetwarza tekst w schemacie rozmowy i produkuje albo odpowiedź tekstową, albo wywołania narzędzi (function-calling). W Qwen3.5 należy jawnie wyłączyć thinking mode w trybie dialogowym robota (zmniejsza latencję i ryzyko pętli generacji).
- **TTS streaming**: generuje audio w chunkach (np. 100–200 ms) i odtwarza praktycznie od razu; w trybie offline TTS (np. Riva offline) czekasz na całość i tracisz UX.

Współbieżność z wizją/propriocepcją:

- Wątek/proces audio powinien mieć priorytet czasowy (soft-RT): stałe ramki, minimalny jitter.
- LLM bywa najbardziej „burzliwy" obliczeniowo (GPU), więc sensowne jest uruchomienie ASR i TTS w trybie CPU-first (Vosk + Piper), a dopiero w trybie „high accuracy" przenoszenie ASR na GPU (faster-whisper).
- Na Orin NX limit mocy/termiki jest realny (10/15/25W), więc dobór profilu i równoległych obciążeń jest częścią projektu.

Detekcja mowy, diarization i progi ufności

VAD:

- **Silero VAD**: MIT, przenośny przez PyTorch i ONNX; repo podkreśla brak telemetrii i „zero strings attached".
- **WebRTC VAD**: bardzo lekki i szybki; py-webrtcvad to interfejs do WebRTC VAD, a dokumentacje społeczności wskazują licencję MIT dla wrappera i BSD-style dla oryginalnego kodu WebRTC.

W praktyce: WebRTC VAD jako „pierwszy filtr", Silero jako „drugi filtr" zwiększający jakość (redukcja fałszywych startów).

Diarization (opcjonalnie):

- Jeśli robot rozmawia zwykle z jedną osobą, diarization może być wyłączona (oszczędność CPU/GPU).
- Jeśli potrzebujesz rozróżniania mówców w rozmowie grupowej, rozważ wykorzystanie sherpa-onnx, który deklaruje wsparcie speaker diarization offline (onnxruntime) w tym samym SDK.
- Alternatywnie, utrzymuj prostą heurystykę „push-to-talk”/„wake word” (jeśli masz) i unikaj diarization w MVP.

Progi ufności i obsługa błędów:

- Dla ASR przechowuj: confidence (jeśli silnik udostępnia), długość wypowiedzi, SNR/poziom RMS, liczbę poprawek w partial.
- Wprowadź progi:
    - CONF_HIGH: automatycznie wykonaj polecenie,
    - CONF_MED: wykonaj *confirm dialog* („Czy chodziło Ci o…?”),
    - CONF_LOW: poproś o powtórzenie i przejdź w tryb bezpieczny (brak ruchu, tylko dialog).
- Dla komend ruchu zawsze wymuszaj confirmation, jeśli komenda dotyczy ryzykownych działań (podniesienie obiektu blisko człowieka, szybki marsz, obrót w tłumie).

Timing i budżet TTFT/TTST

Definicje:

- **TTFT (Time To First Token)**: czas od wysłania promptu do LLM do otrzymania pierwszego tokena.
- **TTST (Time To Start Talking)**: czas od podania tekstu do TTS do rozpoczęcia odtwarzania dźwięku (pierwszy chunk PCM).
- Dla UX robota interesuje Cię „Time to First Audio Response” = (VAD end) + (ASR finalize) + (LLM TTFT) + (TTS TTST).

Na Orin NX 16GB możesz przyjąć realistyczny budżet projektowy (do weryfikacji w testach HIL):

- **VAD/segmentacja**: 10–50 ms obliczeń na sekundę audio (zwykle <5 ms na ramkę 20–30 ms; zależy od implementacji).
- **ASR finalize**: ~0.2–1.2 s dla krótkich komend (zależnie od silnika; Vosk zwykle szybciej i bardziej „online”, Whisper wolniej, ale jakościowo lepiej).
- **LLM TTFT**: ~0.2–0.8 s (dla 2–4B int4, non-thinking, krótki kontekst – estymacja). Jako punkt odniesienia dla generacji: w dyskusji llama.cpp dla Orin NX 16GB podano ok. 9 tok/s dla generacji (test tg128) na CPU dla 7B Q4_0, co wyznacza konserwatywny dolny pułap; GPU zwykle poprawi wynik, ale zależy od runtime i konfiguracji.
- **TTS TTST (Piper**): ~0.05–0.3 s przy streamingu i krótkich buforach (estymacja), zależne od „quality level” i długości tekstu; poziomy jakości i rząd wielkości parametrów są jawne.

Wypadkowo: **0.7–2.5 s do pierwszego dźwięku odpowiedzi** dla typowych poleceń głosowych jest realnym celem.

ROS2/DDS: szkice schematów wiadomości i kontraktów API

Ponieważ G1 używa DDS w warstwie komunikacji (a przewodniki integracyjne wskazują CycloneDDS i kompatybilność z ROS2), najczystsze jest zrobienie audio pipeline jako zestawu węzłów ROS2 publikujących/odbierających zdarzenia, a nie „czarnych skryptów".

Proponowane wiadomości ROS2 (przykładowe, minimalistyczne):

```
text
# pcdb_msgs/msg/AudioFrame.msg
builtin_interfaces/Time stamp
uint32 sample_rate_hz
uint8  channels
string encoding       # "pcm_s16le"
uint8[] data
text
# pcdb_msgs/msg/VadEvent.msg
builtin_interfaces/Time stamp
string event          # "speech_start" | "speech_end" | "speech_continue"
float32 speech_prob   # 0..1 (jeśli dostępne)
text
# pcdb_msgs/msg/AsrPartial.msg
builtin_interfaces/Time stamp
string text
float32 confidence
bool is_final
text
# pcdb_msgs/msg/DialogTurn.msg
builtin_interfaces/Time stamp
string role           # "user" | "assistant" | "system"
string text
float32 confidence    # dla user; dla assistant = 1.0
text
# pcdb_msgs/msg/TtsChunk.msg
builtin_interfaces/Time stamp
uint32 sample_rate_hz
uint8 channels
string encoding       # "pcm_s16le"
bool is_last
uint8[] data
W warstwie LLM zalecam kontrakt w stylu function-calling, ale tylko do
zdefiniowanych „skills":
json
{
  "name": "call_skill",
  "description": "Wywołuje bezpieczny skill robota z walidacją parametrów i
limitami.",
  "parameters": {
    "type": "object",
    "properties": {
```

```
      "skill_id": { "type": "string", "enum": ["say", "navigate_to", "set_posture",
"stop_all", "manipulate"] },
      "args": { "type": "object" },
      "requires_confirmation": { "type": "boolean" }
    },
    "required": ["skill_id", "args"]
  }
}
```

Następnie „Skill Executor” (poza LLM) waliduje `skill_id`, sprawdza limity i wykonuje dopiero bezpieczny zestaw komend do SDK/sterowania (DDS). To podtrzymuje zasadę: **LLM nie steruje bezpośrednio ruchem**.

Diagram pipeline’u mowy

```
mermaid
flowchart LR
  MIC[Wejście audio\nmikrofony robota] --> CAP[Capture + resample\n16kHz PCM]
  CAP --> VAD[VAD\n(WebRTC + Silero)]
  VAD -->|speech_start| BUF[Ring buffer + segmenter]
  BUF --> ASR[ASR streaming\n(faster-whisper / Vosk / whisper.cpp)]
  ASR -->|partial| UI[Dialog UI\n(na ekran/telemetria)]
  ASR -->|final text + conf| NORM[Normalizacja + polityka progów]
  NORM -->|CONF_LOW| REASK[Re-prompt\n\"Powtórz proszę\"]
  NORM -->|CONF_MED| CONFIRM[Dialog potwierdzenia\n\"Czy chodziło o...?\"]
  NORM -->|CONF_HIGH| LLM[LLM lokalny\n(Qwen3.5 2–4B int4)]
  LLM -->|tool calls| GATE[Safety Gate\nschematy+limity+timeout]
  GATE --> SKILL[Skill Executor\nDDS/ROS2]
  LLM -->|assistant text| TTS[TTS streaming\n(Piper / sherpa-onnx / Riva)]
  TTS --> SPK[Wyjście audio\ngłośnik robota]

  PROPRIO[Propriocepcja\ntryb, alerty, temp] --> GATE
  VISION[Wizja\nstreszczenia] --> LLM
```

Integracja bez Internetu: magazyn modeli, cold-start, swapping, I/O i kontenery

Magazyn modeli i struktura wersjonowania

Ponieważ Unitree wskazuje „Storage 2T” w konfiguracji komputera developerskiego, masz przestrzeń na utrzymanie wielu wariantów modeli (ASR/TTS/LLM) i rollout A/B. Jednocześnie Orin NX wspiera NVMe przez PCIe, co jest preferowane dla dużych modeli i szybkiego ładowania.

Praktyczna struktura offline:

- /opt/models/asr/whisper/base-int8-ct2/…
- /opt/models/asr/vosk/pl-small/…
- /opt/models/tts/piper/pl_PL-gosia-medium/…
- /opt/models/llm/qwen35-4b-int4/…
- /opt/models/MANIFEST.json (hashy SHA-256, wersje, licencje, źródła)

W trybie offline kluczowe są „manifesty” i spójny mechanizm „model packów”, bo nie możesz w locie pobierać brakujących tokenizatorów czy plików config.

Cold-start i utrzymanie „ciepłych” procesów

W systemach wbudowanych cold-start jest częstą pułapką. Dla whisper.cpp raportowano przypadki, w których „first load time” na Jetsonach (AGX Xavier/Orin) był rzędu wielu minut i stawał się bottleneckiem produkcyjnym. Nawet jeśli to nie wystąpi w Twojej konfiguracji, w robocie i tak warto:

- uruchamiać ASR i TTS jako **długowieczne demony** (systemd/contener) i utrzymywać modele w pamięci,
- stosować „warm-pool” (1–2 procesy ASR gotowe do pracy) zamiast odpalania procesu per zapytanie,
- ograniczyć liczbę wariantów modeli aktywnych jednocześnie (swapping kontrolowany).

Swapping modeli i zarządzanie pamięcią

Swapping jest potrzebny (tryb jakości vs low-power), ale musi być deterministyczny:

- ASR: przełączaj silnik, nie tylko model (np. Vosk ↔ faster-whisper) w zależności od obciążenia GPU/temperatury.
- TTS: przełączaj głos (Piper: różne quality levels) i fallback na eSpeak NG przy przeciążeniu.
- LLM: przełączaj 2B ↔ 4B w zależności od zadania i czasu reakcji.

W praktyce wybierz strategię:

- „Jedna instancja na raz” + szybki restart (proste, ale ryzyko przerwy),
- lub „dwóch rezydentów” (np. Vosk + faster-whisper, Piper + eSpeak) i routing (bardziej niezawodne, ale koszt RAM).

Kontenery na Jetson w środowisku offline

NVIDIA publikuje przewodniki o uruchamianiu kontenerów na Jetson; Docker jest integralną częścią ekosystemu JetPack, a na Jetsonach używa się NVIDIA Container Runtime.

Tryb offline oznacza:

- budujesz obrazy ARM64 w CI (lub na stacji roboczej z buildx),
- eksportujesz je jako tar (docker save),
- przenosisz nośnikiem do robota i importujesz (docker load),
- a same modele dostarczasz jako „model packs” z podpisem i hashami (patrz sekcja wdrożeniowa niżej).

Uwaga specyficzna dla G1: w przewodnikach sugeruje się, że Jetson jest na dedykowanej płycie Unitree i wymaga właściwego BSP (nie flashować „obrazami zewnętrznymi”), więc aktualizacje systemu trzeba planować „ostrożnie i minimalnie inwazyjnie”.

Optymalizacje ASR/TTS na Jetson i bezpieczeństwo działania offline

Optymalizacje wydajnościowe specyficzne dla Jetson Orin NX

Kwantyzacja i formaty:

- **Faster-whisper + CTranslate2**: korzystaj z int8 (na CPU i GPU) tam, gdzie akceptujesz minimalny spadek jakości; sam projekt wskazuje, że 8-bit quant zwiększa efektywność.
- **whisper.cpp**: używaj quant modeli ggml (repo opisuje wsparcie kwantyzacji i narzędzie quantize).
- **Piper / ONNX**: dobieraj „quality level" pod budżet TTST i naturalność; parametryzacja (x_low…high) jest jawna, co ułatwia tworzenie profili „low-power" i „high-quality".

Streaming i batching:

- W ASR priorytetem jest streaming i małe porcje, a nie duże batche (batche zwiększają throughput, ale pogarszają interakcyjność).
- W TTS w praktyce generuj audio w chunkach (np. 100–200 ms) i odtwarzaj natychmiast; tryb offline-only (generuj wszystko, potem odtwórz) jest UX-owo gorszy. Riva TTS dokumentuje różnicę: offline zwraca audio dopiero po wygenerowaniu całej sekwencji, a streaming zwraca podczas syntezy.

Podział CPU/GPU:

- Jeśli GPU jest krytyczny dla LLM/wizji, ustaw ASR na Vosk (CPU) i TTS na Piper (CPU), co stabilizuje latencję dialogu.
- Gdy GPU ma headroom: przenieś ASR na faster-whisper (CUDA) dla jakości w hałasie.

Profil mocy i throttling:

- Orin NX 16GB ma tryby 10W/15W/25W; w robocie „długie rozmowy" potrafią doprowadzić do throttlingu, więc profil mocy i chłodzenie muszą być częścią walidacji.

Safety i degradacja jakości

Ryzyko specyficzne dla audio w robocie to „złe rozpoznanie → zła akcja". Dlatego:

- Stosuj progi ufności i potwierdzenia opisane wcześniej.
- Dla komend „ruchowych" zawsze stosuj regułę: jeśli ASR nie ma wysokiej ufności, robot **nie wykonuje ruchu**, tylko dopytuje.
- Wprowadź „canned responses" i zachowania regułowe (np. „Nie zrozumiałem. Powtórz.") w przypadkach:
  - przekroczenia time-out LLM,
  - awarii ASR/TTS,
  - przeciążenia termicznego (temperatura).
- Dodatkowo rozważ „tryb ciszy": jeśli VAD wykrywa hałas/krzyki (wysokie RMS), robot ogranicza ruch i prosi o podejście bliżej.

Prywatność audio i bezpieczeństwo systemu offline

Prywatność:

- W trybie offline masz przewagę: audio nie opuszcza robota. Ale nadal musisz zdecydować, czy nagrywasz i logujesz próbki do debugowania.
- Polityka rekomendowana: domyślnie **brak trwałego zapisu surowego audio**; jeśli włączasz nagrania do debug, to:
    - rotacja (np. ostatnie 10–30 minut),
    - szyfrowanie na dysku,
    - dostęp tylko w trybie serwisowym,
    - anonimizacja metadanych.

Bezpieczeństwo infrastruktury:

- Unitree przewodniki zwracają uwagę na ryzyko domyślnych ustawień dostępu (hasła) przy podłączeniu do Internetu; nawet offline warto zmienić domyślne hasła i ograniczyć usługi.
- API między modułami (ASR/LLM/TTS) powinno być lokalne (localhost/ROS2), bez nasłuchiwania na zewnętrznych interfejsach.

Licencje jako element bezpieczeństwa wdrożenia:

- **Piper** ma MIT, ale w społeczności wskazuje się, że część pipeline'u fonemizacji używa espeak-ng (GPL) i trzeba to sprawdzić w Twojej kompozycji zależności.
- **eSpeak NG** jest GPL-3.0+ i wbudowanie go w produkt może wymuszać określone obowiązki dystrybucyjne.
- **Coqui TTS**: kod (toolkit) jest MPL-2.0 i dopuszcza komercję, ale popularne wagi XTTS są pod CPML i wprost ograniczają użycie do niekomercyjnego (oraz społeczność wskazuje brak ścieżki legalnego pozyskania licencji komercyjnej).

Plan testów, metryki, budżety zasobów i wdrożenie offline

Zbiory danych i benchmarki dla języka polskiego

Do oceny ASR w języku polskim (offline, command-and-control + rozmowa) rekomenduję trzy klasy danych:

- **Mozilla Common Voice – Polish**: oficjalne datasheet wskazuje ok. 187.97 h nagrań (175.72 h zwalidowanych), ~3455 mówców w wersji 24.0; Fundacja Mozilla komunikuje, że dataset jest udostępniany na licencji CC0.
- **Spokes (CLARIN-PL/PELCRA)**: korpus rozmów nieformalnych; w CLARIN wiki podano skalę ~1024 h i 658k wypowiedzi; to dobre do testów w dialogu spontanicznym (VAD, segmentacja, overlap).
- **SpokesBiz**: korpus konwersacyjny >650 h, z diarization i ręczną anotacją interpunkcji i casing – szczególnie cenny do walidacji diarization i jakości końcowej transkrypcji w rozmowie.

Do TTS (polski) i/lub walidacji jakości syntezy:

- **M-AILABS Polish**: opis korpusu wskazuje ~53.83 h, ~26,322 wypowiedzi, 2 mówców; to może służyć do trenowania/porównywania lub dopasowania lekkich głosów (choć 2 mówców ogranicza różnorodność).
- Dla gotowych głosów Piper: korzystaj z oficjalnych pl_PL głosów lub utrzymywanych zestawów polskich modeli; w ekosystemie wprost występują polskie modele „vits-piper-pl_PL-…".

Plan testów HIL i metryki

Metryki ASR:

- WER/CER na Common Voice PL i na własnym zestawie komend robota (ok. 200–1000 fraz).
- Real-Time Factor (RTF) i „partial stability" (liczba zmian w partial tekstu na sekundę).
- Odporność na hałas: testy w ruchu, z wentylatorem, w hali, w pobliżu muzyki.

Metryki TTS:

- TTST, RTF syntezy, dropouty audio (glitch count), poziom głośności i clipping.
- Subiektywne MOS (min. 10–20 uczestników), ale w wersji robota ważniejsze jest „zrozumiałość w hałasie" niż „radiowa naturalność".

Metryki end-to-end:

- Time-to-First-Audio-Response (od końca mowy użytkownika do pierwszego dźwięku odpowiedzi).
- Task success rate (czy robot wykonał prawidłowy skill / czy odmówił przy niepewności).
- Safety events: liczba wymuszonych potwierdzeń, liczba „stop_all", liczba odrzuconych tool-calls przez Safety Gate.

Budżet zasobów i latencji na Orin NX 16GB

Poniższe liczby są **zakresami projektowymi** (dla konfiguracji referencyjnej PC2=Orin NX 16GB). Tam, gdzie istnieją twarde dane źródłowe (np. rozmiary whisper.cpp, wymagania Riva), podaję je wprost; reszta to estymacje do weryfikacji w benchmarku HIL.

Etap	Wariant rekomendowany	Footprint pamięci (rząd wielkości)	Przepustowość / latencja (rząd wielkości)	Uwagi
VAD	Silero VAD ONNX + opcjonalnie WebRTC VAD	dziesiątki MB lub mniej (model mały), CPU	opóźnienie na ramkach audio zwykle bardzo niskie; decyzje co 10–30 ms	Silero MIT i przenośność ONNX; WebRTC bardzo lekki.
ASR	Tryb „quality": faster-whisper int8	zależne od modelu; w praktyce od setek MB do kilku GB (large)	celem jest near-real-time dla krótkich komend; projekt deklaruje do ~4× szybciej i mniej pamięci vs openai/whisper, plus int8 na CPU/GPU	Preferuj tiny/base/small dla robota (latencja), large tylko „offline batch".

Etap	Wariant rekomendowany	Footprint pamięci (rząd wielkości)	Przepustowość / latencja (rząd wielkości)	Uwagi
ASR	Tryb „light”: Vosk small PL	~50 MB dla małych modeli (deklaracja)	partial w czasie rzeczywistym; finalize zwykle setki ms po końcu mowy (zależne od akustyki)	Bardzo dobry, gdy GPU jest zajęte.
LLM	Qwen3.5-2B/4B int4, non-thinking	typowo 1–3+ GB na wagi + KV cache (zależnie od limitu kontekstu)	TTFT ~0.2–0.8 s (cel); tok/s zależne od runtime. Konserwatywny punkt odniesienia: llama.cpp na Orin NX 16GB raportuje ok. 9 tok/s generacji na CPU dla 7B Q4_0.	Ogranicz kontekst i wyłącz thinking mode.
TTS	Piper pl_PL medium	dziesiątki MB na model (np. ~63 MB)	TTST typ. 50–300 ms (cel); prędkość zwykle >1× real-time dla prostych głosów (do weryfikacji)	Modele mają poziomy jakości i parametry (x_low…high).
TTS fallback	eSpeak NG	bardzo mały, CPU	praktycznie natychmiastowy TTST	GPL-3.0+.

Wdrożenie offline: CI/CD modeli, podpisy, rollback, monitoring

Model CI/CD (offline-compatible):

- Buduj „model packs” jako archiwa (np. .tar.zst) zawierające: wagi, configi, tokenizatory, licencje, manifest SHA-256.
- Podpisuj pakiety (np. mechanizmem podpisu artefaktów w Twojej infrastrukturze) i waliduj podpis na robocie przed instalacją.
- Instalacja offline: nośnik USB / dysk zewnętrzny / fizyczne repo w sieci zamkniętej.

Rollback:

- Trzymaj co najmniej 2 wersje każdego model packa: current i previous.
- Runtime powinien umieć przełączyć się na poprzedni pakiet bez reflasha systemu (atomowy symlink + restart usługi).

Monitoring audio pipeline:

- VAD: liczba wykryć, false-starts (heurystycznie), średnia długość segmentów.
- ASR: RTF, średnia confidence, odsetek CONF_LOW/CONF_MED.
- LLM: TTFT, tok/s, odsetek odmów i odrzuconych tool-calls przez Safety Gate.
- TTS: TTST, dropouty, opóźnienie buforów audio.

Logowanie prywatnościowe:

- Loguj metryki i skróty (hash) transkrypcji, a nie surowe audio.
- Jeśli przechowujesz audio do debug: szyfrowanie + krótka retencja + tryb serwisowy.

Najważniejsze decyzje architektoniczne, które zwykle przesądzają o sukcesie

- strumieniowanie (partial ASR + streaming TTS) zamiast trybu „batch”,
- twarde progi ufności i dialog potwierdzeń przed akcją,

- rozdział zasobów CPU/GPU między audio i LLM/wizję,
- utrzymywanie „ciepłych” procesów i kontrola cold-start,
- jawne wersjonowanie i podpisy pakietów modeli na aktualizacje offline.